

Recorder bake-off: K vs T

Two small Android apps that do one thing: record GPS for as long as a session runs, with the phone locked in a pocket, and measure how much of the track they kept. They exist to pick the recorder the native golf-tracker will use. **They never feed a round.**

Why a native recorder at all, in Matt's words (docs/HANDOFF-native-build.md, Section 2): *"The real motivation is what you stumbled on and it is 2 fold: allowing the GPS to run in the background and mark the track regardless of what I am doing on the phone and 2 eliminating the phone but not with a watch - remember my background."*

What he decided, 2026-09-13

Decision	His pick
Native shell for the build before 2026-10-07	"Native shell (Recommended)"
Contenders	"Kotlin vs transistorsoft (Recommended)"
Who builds	"Opus builds, Fable reviews (Recommended)": Opus builds this app. Fable reviews it at xhigh before it goes on the course and gives the pass/fail verdict. The recorder that feeds rounds, and its storage, stay Fable's
Pass bar	"99% coverage, no gap > 20 s (Recommended)"
Installs	"You may install whatever is needed"

The two apps

	Bake-off K (teal icon)	Bake-off T (orange icon)
Application id	com.postmaster87.golfbakeoff.k	com.postmaster87.golfbakeoff.t
Recorder	Hand-written Kotlin: fused location, high accuracy, 1 s interval, no distance filter, in a location foreground service holding a partial wake lock	transistorsoft tslocationmanager 4.5.1, debug build
Source	app/src/handwritten/	app/src/transistor/

Everything else is shared code in app/src/main/: the screen, the files, the heartbeat, the measure, the export, and LocationFix.kt, the single path from an Android Location to a row. Both apps show a notification while recording.

Two apps, not one app with two recorders. Each gets its own process, foreground service, notification and battery state. In one app, whichever recorder held a foreground service would keep the other one alive, and the comparison would measure nothing. Carried together on the same round, both face the same phone, pocket and sky.

T: four defaults changed, and nothing else

Each default is changed because transistorsoft's own docs or source say it is wrong for this job:

Default	Why it is wrong here	Set to
The location filter: an accuracy gate (trackingAccuracyThreshold, 100 m) and a policy that can reject outliers, with Kalman on	In the 4.5.1 source (LocationFilter.evaluate), an accepted fix keeps its coordinates, but whole fixes are rejected: any worse than the gate (accuracyThreshold > 0 && accuracyCurrent > accuracyThreshold), and under Conservative, implied-speed and outlier fixes. Kalman smooths the distance between fixes, not the positions. The web docs name Conservative as the default; the 4.5.1 source's own table names Adjust. A rejected fix is a hole in the track	PassThrough, trackingAccuracyThreshold = 0 ("PassThrough does not disable trackingAccuracyThreshold"), useKalman = false
Stop detection on	GPS turns off once he stands still (philosophy), which is exactly when he plays a shot	activity.disableStopDetection = true ("Location services will never turn OFF", ActivityConfig), then changePace(true)

Default	Why it is wrong here	Set to
allowIdenticalLocations = false	"By default, the Android plugin will ignore a received location when it is identical to the previous location." The docs' own example of an identical location is one fix delivered twice. Whether a new fix at the same coordinates also counts is not stated, and the check is not in the published sources. true is the setting that cannot drop a stand	true. The cost: repeats of a fix time are kept, and the tool counts and shows them (see run 2)
distanceFilter = 10 m, fastest interval unset	It samples on distance, not time, and "If not configured, the default fastest interval is 30000 ms"	distanceFilter = 0, locationUpdateInterval = 1000, fastestLocationUpdateInterval = 1000

Debug builds are "fully functional... without a License Key" (licence Section 3.5, [licence](#)). If T wins, whether to buy a licence for his own rounds is his call.

T: two integration rules any app on T needs

These are not tuning. They come from the SDK's EventManager in the 4.5.1 source, and the emulator smoke test found them the hard way (run 1, below):

1. **While headless (no live screen), the SDK delivers only to a class named** <application id>.BackgroundGeolocationHeadlessTask, **never to in-app listeners** (EventManager.deliver). Fixed: app/src/transistor/.../t/BackgroundGeolocationHeadlessTask.kt.
2. **Back on a screen, delivery is buffered until ready() is called again** (EventManager.isDeliverable: "headless -> foreground ... MUST buffer until the client calls ready() again"). Fixed: ready() runs on every onResume.

Both routes write through one recordSdkLocation. The event log records sdk_delivery_route (listener or headless) each time the route changes. On restore, changePace(true) is called only if the SDK did not come back moving by itself.

The measure and the bar

Coverage is the measure behind the table in docs/HANDOFF-native-build.md, Section 9:

- **span**: last fix time minus first fix time, with fix times sorted
- **covered**: the sum of fix-to-fix intervals of 20 s or less
- **gap**: an interval over 20 s; the longest gap is the largest interval
- **median accuracy**: over every fix that carries one

That definition is not recalled. tools/track-coverage.py reproduces all six Section 9 rows from the exports in docs/roundDownloads/ (run 2026-09-13):

Round	Span	Fixes	Covered	Gaps > 20 s	Longest gap	Median accuracy
FT3 Veenker front 9	143.9 min	7,858	105.0 min (73.0%)	16	11.2 min	3.1 m
FT4 Radcliffe 9	180.6 min	11,393	163.9 min (90.8%)	9	5.6 min	3.2 m
FT4 second nine	33.2 min	2,327	33.2 min (100.0%)	0	5 s	3.6 m
FT5 Veenker back 9	129.2 min	9,358	109.1 min (84.5%)	7	6.5 min	3.0 m
FT6 Veenker hole 1	27.7 min	959	14.2 min (51.2%)	3	11.2 min	3.8 m
FT6 Veenker 14-18	75.3 min	5,913	72.0 min (95.5%)	4	1.6 min	3.0 m

The bar: at least 99% covered and no gap over 20 s, over a round-length carry with the screen locked and a music app in use. Under this measure a track with no gap over 20 s is 100% covered, so **the gap clause decides**. One 21 s dropout fails a recorder.

START and STOP count. A recorder that dies before STOP leaves no fixes after it, which the first-to-last measure cannot see; run 1's T log scored as a clean 3.0 min session. So for a session, a first fix more than 20 s after START, or a last fix more than 20 s before STOP (or before the last event, if there is no STOP yet), is an **edge gap**. The gap rule itself is unchanged. Web exports are still scored first fix to last.

T is scored twice: once from its app log, and once from its own SQLite store (the `sdk-store-*.json` that EXPORT writes), over the same START-to-STOP window. A production app on T would read the store. The app log shows whether our integration received what the SDK recorded.

Repeated fix times are 0 s intervals. They cannot change coverage, but they inflate the fix count, so the tool prints how many there are.

The phone (`Coverage.kt`) and the PC (`tools/track-coverage.py`) implement the core measure separately. Both are held to one hand-worked fixture, `app/src/test/resources/coverage-fixture.csv` with `.expected.properties`. The fixture covers an interval of exactly 20 s, a 20.001 s gap, a repeated time, rows out of order, and two broken rows. **Mutation-checked:** changing `>` to `>=` in the gap test fails 2 of the 8 Kotlin tests and 3 of the Python checks. The PC tool's self-test (14 checks) also covers the edge gaps. **The PC tool gives the verdict.** The phone's numbers are only a glance on the course.

What a session records

One START to one STOP. The files are in the app's private storage, and EXPORT copies them to `Download/golf-bakeoff/<K|T>/<session>/`.

- `meta.json`: recorder, app version, device, Android version, start time, the thresholds, and the phone's state at START.
- `fixes.csv`: one row per fix the recorder handed over. Columns are `wall_ms`, `fix_ms`, `elapsed_rt_ms`, `lat`, `lon`, `acc_m`, `alt_m`, `vacc_m`, `speed_mps`, `bearing_deg`, `provider`, `mock`, `moving`, `sample`, `event`. An absent value is an empty cell, never a zero. A fix with no time of its own is not written, and the event log records it instead. `moving`, `sample` and `event` are T's SDK flags. T's intermediate samples are kept and marked; `--exclude-samples` rescores without them.
- `events.csv`: `wall_ms`, `elapsed_rt_ms`, `kind`, `detail`:
 - hb every 5 s: whether the recorder reports itself running, time since the last fix, and the phone's state (screen on, Doze, battery saver, unrestricted, background-restricted, standby bucket, heat, battery, charging, music playing, location on).
 - `previous_exit`: Android's own reason the previous process died (low memory, freezer, signaled, swiped, crash...).
 - `screen_off` / `screen_on` / `unlocked` / `doze_changed` / `battery_saver_changed`.
 - The recorder's own events: `updates_on`, `availability`, `foreground_start_failed`, `sticky_restart`, `resume_on_process_start`, `sdk_started`, `sdk_restored`, `sdk_delivery_route`, `sdk_motionchange`, `sdk_ready_on_resume`, and others.
- `summary.txt`, written at export.
- T only: `sdk-store-<time>.json`, the SDK's own SQLite record, one level up from the sessions.

Each gap gets a description, not a guess at its cause. For every gap, including edge gaps, the PC tool reads the heartbeats inside it:

Heartbeats inside the gap	The tool says
<code>recorder=1</code> all the way through	<i>recorder reported running throughout; no fixes reached the log</i>
None at all	<i>the app was not running</i>
Some, but none from a running recorder for more than 15 s	<i>recorder not reported running for up to N s</i>

It also lists every other event within 5 s of the gap: location turned off, a process death, Doze, a delivery-route change. Those events are the evidence for why no fix arrived; the heartbeat alone cannot say.

Write posture. Each row is flushed to the OS as it is written, so a killed process loses nothing it had handed over, and `fsync` runs every 15 s. A process killed mid-row leaves a cut row. The next process ends that line before appending, and the reader counts the cut row as skipped instead of reading a wild timestamp.

An active session resumes. If the process comes back without its recorder, both apps restart it, because the session says he is recording. The gap stays in the file, and the event log says why.

Verified on the emulator, 2026-09-13

What this is, and what it is not. It ran on an Android 15 emulator (golf-bakeoff, Google APIs x86_64), with synthetic GPS fed one fix a second along a walk from Veenker's first tee (adb emu geo fix), in sessions of 4 to 9 minutes. **n = 3 runs on one emulator.** It proves the plumbing, not the bar: no Samsung app sleeping, no pocket, no sky, no round-length carry. The 5.0 m median accuracy is the emulator's synthetic value, not a phone measurement.

- **Unit tests:** 8/8 in each app (the same 8 tests compiled into both).
- **PC tool self-test:** 14/14.
- **Mutation check:** as above.

Run	What was done	K	T
1 (build 2)	START both; 60 s screen off with Doze forced; 35 s location off; kill -9 both	Kept recording in forced Doze (+55 rows in 61 s). Location off made a 41 s gap, described as "recorder reported running throughout; no fixes reached the log", with availability available=false listed beside it. After the kill, fixes were back within about 1 s (previous_exit reason=signaled, resume_on_process_start, updates_on)	Same through Doze and location off (40 s gap). After the kill, the SDK restored enabled=true moving=true and its own store kept filling (392 to 412 rows in 20 s), but the app log got nothing for 357 s. logcat: 530 times "Attempted to post headless event location but there are no listeners". That is the headless rule. Scored FAIL (2 gaps) once START/STOP edges count
2 (build 3: headless task, ready() on resume)	START both, HOME, 40 s; both screens destroyed with the processes alive ("Don't keep activities"), 40 s; kill -9 both, 60 s; reopen; STOP; EXPORT	PASS: 4.8 min session, 267 fixes, 0 gaps, longest 6 s. Fixes back 1.3 s after the kill	PASS from the app log: 336 fixes, 0 gaps, longest 6 s. PASS from the SDK store: 333 fixes. Delivery went listener, then headless 6.9 s after the kill, then listener again after reopen; 0 "no listeners" warnings
3 (build 4: changePace only if not restored moving)	T only: START, HOME, 25 s; kill -9; 60 s; STOP	-	The restart took about 57 s this time (previous_exit 17:24:58, restore 17:25:55), against about 5 s in run 2: Android's restart backoff for a second kill minutes after the first. Repeated fix times still appeared after the restart (22 in the log, 21 in the store) without our changePace, so they come from the SDK or the provider

Repeated fix times: in run 2, 88 in T's log and 93 in its store, all after the kill; K had 0. Coverage is unchanged, and the fix count is inflated by that many.

What only the phone can answer: Samsung's app sleeping, a round-length carry, the sky at Veenker, and whether Android's restart backoff ever bites when no one is killing the app on purpose.

Putting it on the phone

When is his call: Fable's xhigh review comes before the course; a carry off the course can happen before that.

On your phone (Galaxy S26), before plugging in:

1. Settings → Security and privacy → **Auto Blocker** → **Off**. Auto Blocker blocks sideloading and USB commands (the same step as the memo app's SETUP.md).
2. Settings → Developer options → **USB debugging** → **On**.
3. Plug the phone into the desktop by USB, and tap **Allow** on "Allow USB debugging?".

On the desktop: Claude runs .\bakeoff.ps1 install. Two apps appear: Bake-off K (teal) and Bake-off T (orange).

On your phone, in each app (K, then T):

4. Open it. Under BEFORE YOU START, tap **FIX** on every row that says NO, and allow what Android asks. "Location: allow all the time" opens a settings page: choose **Allow all the time**. For "Battery: unrestricted", choose **Allow**.

5. The Samsung row stays -- because the app cannot read it. Set it by hand: Settings → **Battery** → **Background usage limits** → **Never auto sleeping apps** → + → tick Bake-off K and Bake-off T → **Add**. Samsung's support page words the path as Settings → Battery and device care → Battery → Background usage limits → Never sleeping apps ([Samsung](#)). If your menu reads differently, send a screenshot.

On your phone, for a carry:

6. Open Bake-off K and tap START. Open Bake-off T and tap START.
7. Lock the phone and pocket it for a round's length. Play music and use the phone the way you normally would.
8. At the end, in each app: hold **HOLD TO STOP** for 1.5 s, then tap **EXPORT ALL TO DOWNLOADS**.
9. Plug the phone in. Claude pulls Download/golf-bakeoff into docs/roundDownloads/bakeoff/ and scores it.

Build, test, install, pull

Claude runs these on the desktop. They are here so they can be repeated.

```
.\bakeoff.ps1 test      # JVM unit tests of the measure, both apps
.\bakeoff.ps1 build     # both debug APKs
.\bakeoff.ps1 install   # both APKs onto the one device adb sees
.\bakeoff.ps1 pull      # Download/golf-bakeoff -> docs/roundDownloads/bakeoff (gitignored)
python ..\..\tools\track-coverage.py ..\..\docs\roundDownloads\bakeoff
python ..\..\tools\track-coverage.py --self-test
```

Toolchain:

- compileSdk 36, which transistorsoft's setup page requires.
- Android Gradle plugin 8.13.2, which supports API 36.1 and needs Gradle 8.13.
- Kotlin 2.3.21, whose compatibility table covers AGP 8.13.
- Gradle 8.13, checked against its published SHA-256.
- minSdk 30, for Android's process exit reasons.